Anonymity Preserving, Decoupled Distributed Authentication for Multihop IoT Networks

Sreeparna Das, Anup Kulkarni, Manas Khatua
Computer Science and Engineering
Indian Institute of Technology Guwahati
{d.sreeparna, anup.kulkarni, manaskhatua}@iitg.ac.in

***Abstract*—The Internet of Things (IoT) demands lightweight and secure authentication for resource constrained multihop networks, while also protecting the privacy of individual devices. Many distributed authentication schemes have been proposed in the past for IoT networks. Among those schemes, the decoupled distributed authentication scheme (DAuth) has been proposed recently for multihop IoT networks. However, we have identified two critical gaps in that scheme. First, the real device identity is exposed on the open channel in every authentication session, enabling adversarial cross-session tracking. Second, packet loss in the session key establishment phase desynchronises the device and server states, forcing full re-enrolment. Therefore, this paper proposes a security improved and anonymity preserving decoupled distributed authentication scheme, addressing both the identified gaps. Device anonymity is preserved through per-session pseudonym rotation, where a rotating pseudonym replaces the real device identity in all open-channel communications, preventing long-term tracking. A dual-state synchronisation mechanism maintained at both the authentication server and the gateway/root enables seamless recovery from single packet-loss events without re-enrolment. The proposed scheme is evaluated via ProVerif-based formal security analysis, and theoretical and simulation-based performance analysis confirm that the proposed scheme reduces energy by 42.8 % and 32.4 %, and CPU time by 42.7 % and 32.4 %, over the state-of-the-art LAAKA and Zhou et al. schemes, respectively, while maintaining the lowest overhead among all compared schemes.**

***Index Terms*—IoT Authentication, Distributed Authentication, Anonymity, Desynchronization in Authentication, Dual-State Storage.**

I. Introduction

The Internet of Things (IoT) connects millions of smart devices that collect and transmit data to centralized platforms [1] such as gateways, often through multihop paths [2]. Industrial and outdoor deployments expose devices to a range of adversarial threats, making authentication a prerequisite for any communication session. Because IoT nodes are battery-powered and resource-constrained, authentication schemes must be lightweight as well as secure [3].

Centralized schemes route all authentication traffic through a gateway, imposing excessive relay load on intermediate nodes in a multihop topology [4] [5] [6]. Distributed schemes distribute this responsibility [7]. However, a plain parent centric distributed authentication schemes overload whichever node a set of new devices happened to join under, while all other capable nodes remain idle [7].

DAuth [8] addressed this skewed-load problem by proposing a lightweight decoupled distributed authentication scheme in which the gateway or root node pre-assigns any capable IoT node (parent or non-parent) as the authentication server for newly joined devices. By decoupling authentication from network joining process, the scheme distributes the authentication load evenly across the network. Authentication process relies on Physically Unclonable Function (PUF)-based secret derivation, a hash-based one-way accumulator and XOR-masked message exchanges, resulting in an efficient and lightweight authentication protocol.

Despite its efficiency, DAuth [8] leaves two security gaps unaddressed. First, the real device identity is transmitted in plaintext over the open channel in every authentication session, enabling an adversary with radio access to track a device across sessions, a direct violation of modern privacy requirements. Second, both the authentication server and the device maintain only a single session state and so a packet loss during session key delivery desynchronises their states, causing all subsequent authentication attempts to fail until the device undergoes full re-enrolment, which is an energy-costly operation for battery-powered constrained nodes.

This paper addresses both gaps with two lightweight extensions to DAuth. To close the anonymity gap, the real identity is never placed on the open channel after enrolment, it is replaced by a rotating pseudonym that is refreshed after every key exchange, so successive sessions become unlinkable to an observer. To eliminate the re-enrolment penalty caused by desynchronisation, both the authentication server and the gateway maintain a dual state, if the session-key update is lost in transit, the device still arrives under its previous pseudonym, which the server recognises and uses to re-synchronise without repeating enrolment.

This paper makes the following key contributions.

1) A anonymity preserving scheme is proposed that introduces per-session pseudonym rotation to preserve device anonymity and a dual-state mechanism to recover from desynchronisation without re-enrolment.
2) The security of the proposed scheme is validated through both informal analysis and formal verification using ProVerif.
3) The efficiency of the proposed scheme is evaluated using both theoretical and simulation-based (COOJA/Contiki-

NG) frameworks.

Section II elaborates on related literature. Section III presents the proposed scheme. Furthermore, Section IV contains the detailed security analysis. Finally, Section V presents the performance analysis, with Section VI stating the conclusion.

II. Literature Survey

Most traditional centralized authentication schemes [4]–[6] rely on a centralized authenticator, leading to increased computational and communication overhead in multihop networks. Distributed authentication schemes mitigate this drawback through multiple authenticators. However, certain distributed schemes [9] suffer from skewed authentication load on particular edge or fog servers and ignore multihop IoT considerations. In LightSAE [7], parent IoT nodes authenticate child nodes, causing skewed authentication overhead on nodes with many children, while those with fewer or no children remain underutilized. On the other hand, traditional peer-to-peer (P2P) schemes depend on long-term key storage, introducing security vulnerabilities. PUF-based P2P schemes [10], [11] mitigate key storage but still suffer from skewed authentication overhead on certain IoT nodes. Both [11], [12] incur high computational and communication overhead due to ECC operations.

DAuth [8] addresses these drawbacks by proposing a lightweight decoupled distributed authentication scheme in which both parent and non-parent nodes can authenticate newly joined devices, mitigating the skewed load scenario. It uses PUF-based secret derivation and a hash-based one-way accumulator for membership verification without revealing device secrets. However, the real device identity is transmitted in plaintext over the open channel in every authentication session, enabling an adversary to track a device across sessions. Additionally, both the authentication server and the device maintain only a single session state, so a packet loss during session key delivery desynchronises their states, forcing full re-enrolment. Zhong et al. [13] present a systematic taxonomy of lightweight anonymous IoT authentication, deconstructing existing protocols into nine sub-frameworks (SF1-UA through SF9-FS) covering individual security and privacy properties such as user anonymity, untraceability, and forward secrecy. These sub-frameworks are then composed into two general frameworks: GF1, which achieves user anonymity but cannot simultaneously provide forward secrecy and desynchronisation resistance; and GF2, which satisfies all three properties together. Their analysis demonstrates that this combination is absent from prior lightweight schemes, establishing GF2 as the target architectural pattern for a complete solution. LAAKA [14] and Zhou et al. [15] address anonymity in IoT authentication but rely on computationally intensive operations, incurring significantly higher overhead than DAuth.

Therefore, this paper proposes an anonymity-preserving extension to DAuth, incorporating two lightweight mechanisms. Per-session pseudonym rotation replaces the real device identity on the open channel with a refreshed pseudonym after every key exchange, making successive sessions unlinkable to an observer. A dual-state mechanism maintained at both the authentication server and the gateway enables seamless recovery from single packet-loss events without re-enrolment.

TABLE I Features Comparison of Authentication Schemes.

Scheme	Dec.	PUF	Acc.	ASU	Anon.	Desync	C.OH	M.OH
RAAF-MEC [9]	×	✓	×	✓	×	×	H	L
SAKMS [12]	✓	×	×	×	×	×	H	L
Li et al. [11]	✓	✓	×	✓	×	×	H	L
Zheng et al. [10]	✓	✓	×	✓	×	×	M	L
DAuth [8]	✓	✓	✓	✓	×	×	L	L
Proposed	✓	✓	✓	✓	✓	✓	L	L

Dec.: Decoupled design. *Acc.*: Hash-based accumulator. *ASU*: Auth. secret update. *Anon.*: Device anonymity via pseudonym. *Desync*: Desync. recovery. *C.OH*: Computation — L (Hash/XOR/$\leq$ 2 AES), M ($>$ 2 AES), H (ECC/Pairings). *M.OH*: Message exchanges — L (1–3), M (4–5), H ($>$ 5).

As shown in Table I, the proposed scheme is compared with representative state-of-the-art schemes. Existing lightweight schemes trade off at least one of: decoupled design, PUF-based authentication, accumulator-based membership verification, anonymity, or desynchronisation recovery. The proposed scheme satisfies all criteria simultaneously while maintaining the lowest computational and communication overhead among all compared schemes.

III. Proposed Scheme

A. System Model and Assumptions

The proposed scheme is designed to help nodes establish secure multihop communications with the gateway. To establish a secure communication, nodes should be authenticated by a trusted authentication server. A set of existing IoT nodes inside the network is entrusted with the responsibility of authentication by the gateway in a distributed manner. The gateway also shares a secret key with each of those responsible IoT nodes/authentication servers. The authentication server can be a parent or a non-parent node, which may be *decoupled*, preventing newly joined child nodes from overwhelming their parent with biased authentication load.

IoT nodes are assumed to be embedded with a PUF and are capable of performing various cryptographic operations, such as encryption/decryption, XOR, hashing, bitwise AND, and concatenation.

Nodes D and AS are assumed to be the newly joined node and its authentication server, respectively. GW represents the gateway symbolically. AS may be located at a singlehop or multihop distance from D. It is assumed that GW has already selected the authentication server set and has established a shared secret key with each member of the set. Let $\mathcal{AS}' = \{AS_1, AS_2, \ldots, AS_n\}$ be the set of authentication servers selected by GW, such that $AS \in \mathcal{AS}'$. GW has a shared secret key $K_{GW\text{-}AS}$ with AS. It is also assumed that D, whose distinct identifier is ID_D, possesses the address of its authentication server AS along with the authentication server's unique identity ID_{AS}. Table II elaborates symbols and functions incorporated in the following scheme.

TABLE II NOTATION.

Symbol	Description
D, AS	Newly joined device and its authentication server
GW	Gateway / root node
$H(\cdot)$	Collision-resistant hash function (SHA-256)
$\mathsf{SE}(\cdot)$, $\mathsf{SD}(\cdot)$	Symmetric encryption/decryption (AES-128)
$\oplus$	Bitwise XOR operation
&	Bitwise AND operation
c_D, $c_{AS\text{-}D}$	PUF challenges for D (issued by AS) and AS (by D)
m_D	Session-based random issued by AS during enrolment
m_{curr}, m_{old}	Current and previous session-based randoms at D and AS
y_D, R_D	Unique secret and PUF response of D
Y_D^H	Hashed unique secret: $H(y_D)$
T_{Acc}	Accumulated authentication value at AS
$\Phi_{AS\text{-}D}$	PUF-response binding: $\mathsf{PUF}_{AS}(c_{AS\text{-}D}) \oplus R_D$
PID_{curr}, PID_{old}	Current and previous pseudonyms of D
$K_{GW\text{-}D}$	Session key between GW and D
$K_{GW\text{-}AS}$	Long-term shared key between GW and AS
ts_1, ts_2, ts_{auth}	Timestamps for freshness verification

There are mainly three phases: node enrolment, node authentication, and node session key exchange. The following sections elaborate on these phases.

B. Node Enrolment Phase

The node enrolment phase is the first phase undergone by a newly joined node D with its authentication server AS. Before entering this phase, it is assumed that D has the required information regarding AS: the address and unique identity (ID_{AS}) of AS. AS can lie at a single-hop or multi-hop distance from D. This phase is assumed to be executed in a secure environment by establishing an end-to-end encrypted channel between D and AS. Every newly joined node executes this phase once until it disconnects from the network. The main purpose of this phase is to enrol the unique secret of D (i.e., y_D) with the accumulated value (T_{Acc}) of AS. During the node authentication phase, D uses y_D again to authenticate with AS. PUF responses and a session-based random are also communicated between AS and D during this phase to establish the secure communication of y_D during authentication. Figure 1 outlines the methodology of this phase.

D initiates the phase by sending a registration request along with its unique identity ID_D to AS. As a response, AS generates a challenge c_D from the predefined challenge set $\mathbb{C}_D$ and a session-based random m_D; these are then forwarded securely to D. Both of these credentials act to secure the communication during authentication: c_D enables D to regenerate its PUF response, while m_D introduces unpredictability into subsequent open-channel messages, preventing adversarial prediction. With c_D as input to its embedded PUF_D, D obtains a unique PUF response R_D. After this, D generates its own unique secret value y_D, which is also a random number; the security of the entire scheme critically depends on maintaining the confidentiality of y_D. D also picks a reverse challenge $c_{AS\text{-}D}$ from the predefined challenge set $\mathbb{C}_{AS\text{-}D}$ and sends $(y_D, R_D, c_{AS\text{-}D})$ back to AS through the secure channel, maintaining the confidentiality of y_D. D then stores $\{y_D, c_D, m_{\text{curr}}\}$. Unlike DAuth [8], which stores m_D as a plain session random, the proposed scheme retains it as the initial *current* session random m_{curr}, seeding the pseudonym mechanism described below.

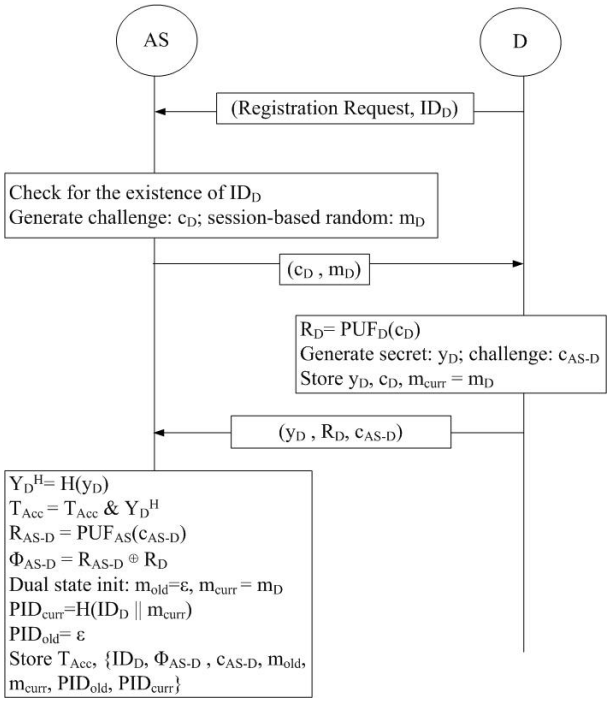


Fig. 1. Enrolment Phase via secure channel.

AS hashes the secret y_D using $H(\cdot)$, generating a tag value Y_D^H. After that, AS performs a bitwise AND operation between the calculated tag Y_D^H and the present accumulated value T_{Acc}, updating T_{Acc} with the result. The accumulated value contains the compact, hashed, unique secrets of individual nodes authenticated by AS. In this way, D successfully enrols itself with AS. Further, AS computes its own PUF response $R_{AS\text{-}D}$ by evaluating PUF_{AS} on the received challenge $c_{AS\text{-}D}$, and calculates $\Phi_{AS\text{-}D} = R_{AS\text{-}D} \oplus R_D$ by operating XOR between $R_{AS\text{-}D}$ and the received R_D. In this way, R_D is kept in XORed form for further secure communications by AS. At the end, AS creates an entry against ID_D and stores the calculated $\Phi_{AS\text{-}D}$ along with the challenge $c_{AS\text{-}D}$ and the received session-based random m_D.

The proposed scheme extends this base enrolment with a pseudonym and dual-state initialisation to eliminate ID_D from all future open-channel messages. Since ID_D is transmitted only over this secure enrolment channel and must never appear on the open channel again, D will henceforth be identified by the rotating *pseudonym* PID, computed as the hash of ID_D concatenated with m_{curr}, in all future authentication messages. Because H is one-way and m_{curr} is a private session random, successive pseudonyms are computationally unlinkable by a channel observer. AS pre-computes PID_{curr} as the hash of ID_D concatenated with m_{curr}, and sets $PID_{\text{old}} = \varepsilon$, $m_{\text{old}} = \varepsilon$, forming the initial *dual state*. This dual state (m_{curr}, m_{old}, PID_{curr}, PID_{old}) is maintained to handle

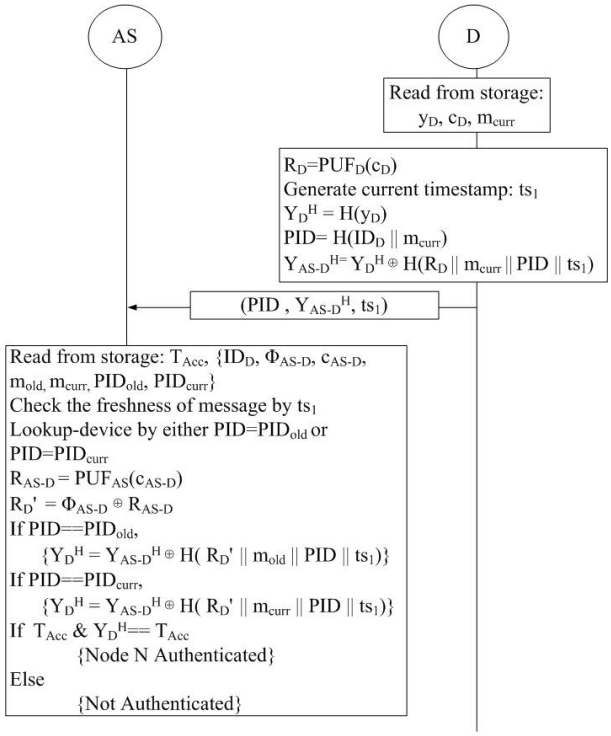


Fig. 2. Authentication Phase via open channel.

desynchronisation: if the updated m_{new} delivered in the Key Exchange Phase is lost in transit, AS can still recognise D by its previous pseudonym and restore synchronisation without requiring re-enrolment. AS therefore stores $\{T_{\mathsf{Acc}}, ID_D, \Phi_{AS\text{-}D}, c_{AS\text{-}D}, m_{\text{curr}}, m_{\text{old}}, PID_{\text{curr}}, PID_{\text{old}}\}$.

C. Node Authentication Phase

The node authentication phase is the second phase of the proposed scheme. Its main purpose is to authenticate D's unique secret y_D with AS. Since this phase is executed over an open channel, y_D must be communicated in secure form, preventing it from being intercepted by adversaries. This security is established through PUF-based challenge-response, producing a secret key R_D, and the session-based random m_{curr} incorporating unpredictability. Both R_D and m_{curr} are known only to D and AS. Figure 2 illustrates the phase.

The phase is initiated by D. D regenerates its PUF response R_D by applying the stored challenge c_D to PUF_D. D then computes its current pseudonym PID by hashing ID_D concatenated with m_{curr}, which is used in place of the real identity ID_D in the outgoing message, unlike DAuth [8], which sends ID_D directly, enabling cross-session tracking. D produces the hashed secret Y_D^H in XORed form as $Y_{AS\text{-}D}^H = H(y_D) \oplus H(R_D \| m_{\text{curr}} \| PID \| ts_1)$. The XOR mask is the hash of R_D regenerated from challenge c_D, the session-based random m_{curr}, the pseudonym PID, and the current timestamp ts_1. The timestamp is used to provide replay protection to the XORed value $Y_{AS\text{-}D}^H$. The complete message $(PID, Y_{AS\text{-}D}^H, ts_1)$ is then sent to AS. An attacker will not be able to obtain $H(y_D)$ from $Y_{AS\text{-}D}^H$ without knowing R_D (which can only be generated by D's PUF) and m_{curr} (known only to D and AS through enrolment).

After receiving the message, AS checks for the existence of an entry matching the received PID. In a departure from DAuth [8], AS performs a dual-state lookup: it checks whether the received PID matches PID_{curr} or PID_{old} in its records. A match on PID_{old} indicates that D missed the Key Exchange Phase delivery in the previous session and still holds the prior m_{curr}; in this case, AS uses the stored m_{old} for the subsequent verification, recovering from the desynchronisation without requiring re-enrolment. AS verifies the freshness of ts_1. R'_D is calculated as $\Phi_{AS\text{-}D} \oplus \mathsf{PUF}_{AS}(c_{AS\text{-}D})$, where both $\Phi_{AS\text{-}D}$ and $c_{AS\text{-}D}$ are retrieved from D's enrolment record. AS regenerates the same XOR mask and unmasks $Y_D^{H'}$, then performs the bitwise AND check by computing a fresh accumulator value $T_{\mathsf{Acc}}^{\mathsf{new}}$ as the bitwise AND of the existing T_{Acc} and the recovered $Y_D^{H'}$, and verifies that it equals T_{Acc}. From the properties of the hash-based one-way accumulator, if an element is combined into an accumulated value using bitwise AND and the same operation is again performed between them, it gives back the same accumulated value. If $T_{\mathsf{Acc}}^{\mathsf{new}}$ equals T_{Acc}, it proves that the unique secret of D is already a legitimate member of AS. With the successful operation, the authentication of D is completed by AS, and AS proceeds to the Key Exchange Phase.

D. Node Session Key Exchange Phase

This phase is executed by AS to inform GW about the authentication of the newly joined node D at the end of the node authentication phase. AS uses an $\mathsf{auth\_token}_D$ to deliver the session key $K_{GW\text{-}D}$ to GW, facilitating session key generation between GW and D. The proposed scheme extends DAuth [8] by additionally deriving a fresh pseudonym $PID_{\text{new}} = H(ID_D \| m_{\text{new}})$ at the end of each session and carrying it within $\mathsf{auth\_token}_D$, so that GW can update its pseudonym mapping for D and successive sessions remain unlinkable. Figure 3 shows the phase.

$K_{GW\text{-}D}$ is calculated by AS by hashing the concatenation of R'_D and m_{new}. R'_D is the PUF response of D regenerated by AS during the node authentication phase, and thus known only to AS and D. m_{new}, on the other hand, is computed as the hash of a fresh random number n_1 generated by AS. The computed m_{new} will also act as the updated session-based random for the next authentication session between AS and D, incorporating an update to the authentication secret. Further, AS derives PID_{new} by hashing the concatenation of ID_D and m_{new}. Because PID_{new} is based on the fresh m_{new}, it is computationally unlinkable to the pseudonym used in the current session. The session key $K_{GW\text{-}D}$, the identities ID_{AS} and ID_D, the next pseudonym PID_{new}, and the authentication completion timestamp ts_{auth} are encrypted under the shared key $K_{GW\text{-}AS}$ (retrieved from storage) to form $\mathsf{auth\_token}_D$. AS simultaneously sends (m_H, ts_2) to D and $(PID_{\text{new}}, ID_{AS}, \mathsf{auth\_token}_D)$ to GW. It then saves the current nonce and pseudonym as the previous values and

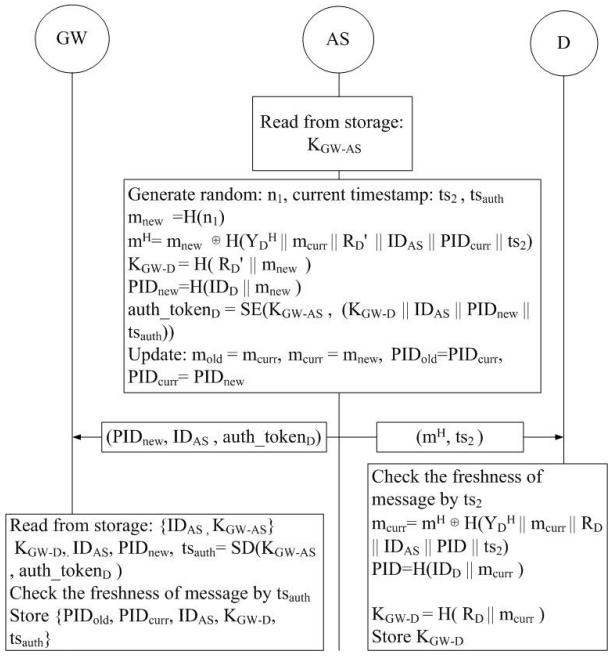


Fig. 3. Session Key Exchange Phase via open channel.

advances its records by setting m_{curr} to m_{new} and PID_{curr} to PID_{new}, while m_{old} and PID_{old} retain the superseded values.

On the other hand, AS also informs D of m_{new} so that D can independently compute the session key $K_{GW\text{-}D}$ and its next pseudonym. Since m_{new} must be protected from adversaries (its capture would allow prediction of future session key material), AS masks it as $m_H = m_{\text{new}} \oplus H(Y_D^{H'} \| m_{\text{curr}} \| R_D' \| ID_{AS} \| PID_{\text{curr}} \| ts_2)$. Here, $Y_D^{H'}$ is the hashed form of the unique secret of D, known to AS from the authentication phase. Both m_{curr} and R_D' are likewise known to AS from the authentication phase. The current timestamp ts_2 is used to prevent replay of m_H in a future session.

Obtaining (m_H, ts_2), D first verifies the freshness of ts_2. D then recovers m_{new} by regenerating the hash mask using its locally held Y_D^H, R_D, m_{curr}, ID_{AS}, PID_{curr}, and the received ts_2, and XORing with m_H. The input of the regenerated hash is already known by D. In this way, D and AS update their shared session-based random m_{curr} with m_{new}. Alongside, D establishes the session key $K_{GW\text{-}D}$ as the hash of the concatenation of R_D and m_{new}, and rotates its pseudonym PID_{curr} by hashing ID_D with m_{new}, thus preparing itself for secure communication with GW.

GW decrypts the token with $K_{GW\text{-}AS}$, verifies ts_{auth}, and extracts $K_{GW\text{-}D}$ and PID_{new}. GW saves PID_{curr} as the previous pseudonym, sets PID_{new} as the current one, and stores the session key against both. Retaining PID_{old} ensures that if the delivery (m_H, ts_2) to D was lost, GW can still service D's data frames under the prior session key pending re-authentication.

E. Desynchronisation Recovery

During the Key Exchange Phase, AS simultaneously dispatches two packets: $(PID_{\text{new}}, ID_{AS}, \texttt{auth\_token}_D)$ to GW and (m_H, ts_2) to D. Loss of either packet leads to a different desynchronisation scenario, each handled without re-enrolment.

If the packet from AS to GW is lost, D has already received (m_H, ts_2) and updated its state to PID_{new} and $K_{GW\text{-}D}$. GW, however, never received PID_{new} or the session key, so it rejects D's subsequent data frames. D detects the failure and re-initiates Phase 2 (Authentication) with its current PID_{curr}. Since D and AS are already synchronised, AS recognises PID_{curr} on the normal path, completes authentication, and forwards a fresh $\texttt{auth\_token}_D$ to GW, restoring normal operation in one additional round.

The primary desynchronisation scenario arises when the packet from AS to D (m_H, ts_2) is lost in transit. AS has already advanced its internal state by updating m_{curr} to m_{new} and rotating PID_{curr} to PID_{new}, while D retains the stale m_{curr}. On the next authentication attempt, D computes its pseudonym from the stale nonce, producing a PID that no longer matches any record at AS. The XOR mask used to protect $Y_{AS\text{-}D}^H$ is likewise constructed from the stale m_{curr}, so AS cannot reconstruct the correct mask and the accumulator check fails. Authentication is therefore rejected. In schemes without a recovery mechanism, this failure forces D to execute the full enrolment phase again, a costly operation for a battery-powered node that re-exposes ID_D on the secure channel and resets all session state.

The proposed dual-state mechanism avoids this re-enrolment cost. If the Key Exchange Phase delivery (m_H, ts_2) is lost, D retains its stale m_{curr}, so the pseudonym it computes in the next session equals PID_{old} from AS's perspective. Algorithms 1 and 2 show how the scheme adapts the Authentication and Key Exchange Phases to recover without re-enrolment. D's behaviour is *identical* in both paths: it always uses its own m_{curr} and PID. Only AS adapts: the dual-state lookup detects which nonce D is on, then constructs m_H with the matching $(m_{\text{old}}, PID_{\text{old}})$ so that D's symmetric decryption succeeds and the states converge. The dual-state mechanism tolerates a single consecutive Key Exchange Phase loss, but two or more consecutive losses exhaust both slots, requiring re-enrolment, a deliberate trade-off between storage overhead and recovery depth.

Algorithm 1 Desynchronisation Recovery – Authentication Phase

Precondition: D holds stale $m_{\text{curr}} = m_{\text{old}}$; AS holds $(m_{\text{curr}}, m_{\text{old}}, PID_{\text{curr}}, PID_{\text{old}})$

$PID = H(ID_D \| m_{\text{curr}})$ ▷ stale m_{curr} gives $PID = PID_{\text{old}}$

D sends $(PID, Y^H_{AS\text{-}D}, ts_1)$ to AS

if $PID == PID_{\text{curr}}$ **then**

    AS verifies using m_{curr} ▷ normal path

else if $PID == PID_{\text{old}}$ **then**

    AS verifies using m_{old} ▷ desync detected

else

    AS rejects D

end if

Algorithm 2 Desynchronisation Recovery – Key Exchange Phase

Precondition: desync path from Algorithm 1; AS uses m_{old}, PID_{old}

$n_1 = \mathsf{rand}()$

$m_{\text{new}} = H(n_1)$

$m_H = m_{\text{new}} \oplus H\left(Y_D^{H'} \| m_{\text{old}} \| R'_D \| ID_{AS} \| PID_{\text{old}} \| ts_2\right)$

AS sends (m_H, ts_2) to D

$m_{\text{new}} = m_H \oplus H\left(Y_D^H \| m_{\text{curr}} \| R_D \| ID_{AS} \| PID \| ts_2\right)$ ▷ D uses $m_{\text{curr}} = m_{\text{old}}$ — recovers correctly

$m_{\text{old}} = m_{\text{curr}}$; $m_{\text{curr}} = m_{\text{new}}$; $PID = H(ID_D \| m_{\text{curr}})$

Result: D and AS re-synchronised; PID rotated

IV. Security Analysis

A. Formal Security Analysis

The proposed scheme is translated into ProVerif [16] queries under the Dolev–Yao adversary model [17], where the adversary has full control of the public channel and can intercept, replay, modify, and inject messages. Ten queries are verified and all are satisfied. Figure 4 shows the verification summary.

1) Protocol Correspondence: Five correspondence queries verify that the protocol steps happen in the correct order and that no adversary can skip or forge a step.

The first query checks enrolment integrity: a device can only be registered at the authentication server after it has explicitly initiated the enrolment process. This rules out any possibility of a device being enrolled without its own participation.

The second query checks the device side of the authentication phase: the device receives the server's acknowledgement and the fresh timestamp ts_2 only after it first sent a valid authentication request. This prevents an adversary from replaying a stale acknowledgement to trick the device into advancing its state.

The third query checks the server side of the authentication phase: the authentication server completes its processing only after it received a structurally correct request from the device. A forged or replayed message cannot push the server into a completed state.

The fourth query binds the Authentication and Key Exchange phases: the PUF response R_D and the session seed m_{curr} that the device commits during authentication are proven to be exactly the same values used in the key exchange. This rules out an active adversary who might try to substitute credentials between phases.

The fifth query covers key exchange: the device recovers the new session seed m_{new} only after it legitimately sent the key-exchange request. No key material can be delivered to the device out of sequence.

2) Secrecy and Forward Secrecy: Two queries verify the secrecy of the session key $K_{GW\text{-}D} = H(R_D \| m_{\text{new}})$ from the perspective of the device and the gateway independently. Even though the adversary observes all traffic on the public channel, it cannot reconstruct $K_{GW\text{-}D}$ because the PUF response R_D is never transmitted and m_{new} is always delivered XOR-masked.

A separate query confirms forward secrecy: the rotated session seed m_{new} itself remains secret. This means that even if an adversary somehow recovers a current session key, it gains nothing about future sessions, because each m_{new} is derived from an independently generated fresh nonce.

3) Anonymity Preservation: The most significant secrecy result concerns the real device identity ID_D. After enrolment, ID_D is never sent over the open channel again. Every subsequent message instead carries the pseudonym $PID = H(ID_D \| m_{\text{curr}})$. ProVerif formally confirms that no attacker, even one who intercepts and manipulates every single message on the network, can recover ID_D from the observed traffic.

This result supports device anonymity. However, anonymity is only meaningful if the pseudonym used in one session cannot be linked to the pseudonym used in the next. The forward-secrecy query addresses this: because m_{new} is proven secret, consecutive pseudonyms $H(ID_D \| m_{\text{curr}})$ and $H(ID_D \| m_{\text{new}})$ are computationally indistinguishable to any observer, making it difficult for an adversary who collects pseudonyms from multiple sessions to determine whether they belong to the same physical device.

4) Offline Guessing Resistance: Finally, a weak-secrecy query confirms that the session key $K_{GW\text{-}D}$ cannot be guessed from the public transcript even by an adversary with unlimited offline dictionary power. This closes the gap against brute-force and dictionary attacks that target the key directly from captured packets.

B. Informal Analysis

1) Replay Attack Resistance: The Enrolment Phase is executed over a secure channel and immune to replay. The Authentication and Key Exchange Phases embed timestamps (ts_1, ts_2, ts_{auth}) and the session-based random m_{curr} into the hash masks. A replayed message with a stale timestamp or stale m_{curr} will fail the freshness check at AS, D, or GW respectively.

2) MITM Attack Resistance: The Enrolment Phase is conducted over an encrypted channel, which precludes MITM. In the Authentication and Key Exchange Phases, the PUF response R_D (device-specific, unreproducible) and session

```
--------------------------------------------------------------
Verification summary:

Query inj-event(DeviceEnrolled(id_D_2,id_AS_3)) ==> inj-event(DeviceEnrollmentStarts(id_D_2,id_AS_3)) is true.

Query inj-event(DeviceAuthenticated(id_D_2,id_AS_3)) ==> inj-event(DeviceAuthenticationStarts(id_D_2,id_AS_3)) is true.

Query inj-event(AuthenticationServerEnds(id_AS_3,id_D_2)) ==> inj-event(AuthenticationServerStarts(id_AS_3,id_D_2)) is
true.

Query inj-event(AuthenticationEndsFull(id_D_2,id_AS_3,R_D_2,m_D_2)) ==> inj-event(AuthenticationStartsFull(id_D_2,id_AS
_3,R_D_2,m_D_2)) is true.

Query event(DeviceKeyExDone(id_D_2,id_AS_3)) ==> event(DeviceKeyExStarts(id_D_2,id_AS_3)) is true.

Query not attacker(SecretK_GW_D_Device[]) is true.

Query not attacker(SecretK_GW_D_GW[]) is true.

Query not attacker(SecretM_New[]) is true.

Query not attacker(SecretID_D[]) is true.

Weak secret SecretK_GW_D_Device is true.
```

Fig. 4. ProVerif Output.

random m_{curr} (private to D and AS) are embedded in every XOR mask. Without R_D and the current m_{curr}, no adversary can forge a valid masked tag $Y^H_{AS\text{-}D}$ or unmask m_H.

3) Device Anonymity: ID_D is transmitted only once, over the secure enrolment channel, and is never exposed on the open network again. All open-channel messages use $PID = H(ID_D \| m_{\text{curr}})$, which is refreshed to $H(ID_D \| m_{\text{new}})$ after every key exchange. Because m_{new} is derived from a freshly generated nonce n_1, successive pseudonyms are computationally unlinkable.

4) Desynchronisation Recovery: Packet loss can occur at three distinct points in the Authentication and Key Exchange Phases, each handled without requiring re-enrolment.

If the acknowledgement and fresh timestamp ts_2 sent by AS to D at the end of the authentication phase are lost in transit, D does not advance its internal state and retains m_{curr} and the current PID unchanged. Since AS has not yet performed any state update at this stage, both entities remain consistent. Upon timeout, D retransmits the authentication request with the same PID and ts_1, and the exchange proceeds normally.

The primary desynchronisation scenario arises when the Key Exchange Phase response carrying the masked session seed m_H, sent from AS to D, is lost. In this case, AS has already advanced its state to m_{new} and rotated the pseudonym to PID_{new}, while D retains the previous m_{curr}. On the subsequent authentication attempt, D constructs its pseudonym from the stale m_{curr}, producing PID_{old}. AS recognises this via the dual-state lookup — storing both $(m_{\text{curr}}, PID_{\text{curr}})$ and $(m_{\text{old}}, PID_{\text{old}})$ — and resumes from the consistent prior state, restoring synchronisation without any re-enrolment overhead.

If the token forwarded from AS to GW is lost, GW does not receive PID_{new} or the associated session key $K_{GW\text{-}D}$. Consequently, D's subsequent data packet, carrying PID_{new}, is not recognised by GW. D re-initiates from the Authentication Phase; the dual-state storage at AS ensures that this re-authentication succeeds and a fresh token is forwarded to GW, restoring normal operation.

5) Forward Secrecy: $K_{GW\text{-}D} = H(R_D \| m_{\text{new}})$. Compromise of a session key does not reveal previous session keys because m_{new} is independently generated from a fresh nonce and R_D is PUF-derived.

V. Performance Analysis

A. Theoretical Performance Analysis

1) Computational Cost: The computational cost is a crucial deterministic factor ensuring the security and efficiency of an authentication scheme. Pycrypto, a Python cryptographic library, is used in a Windows system to measure the computational cost of core cryptographic functions, such as SHA-256 (T_{hash}), AES Encryption/Decryption (T_{aes}), and Random generation (T_{rand}). The PUF operation cost (T_{puf}) is taken from prior benchmarks on the same platform [8].

To execute the node enrolment phase (Figure 1):

- AS computes two random generations (T_{rand}), one PUF operation (T_{puf}), and two hash operations (T_{hash}) for pseudonym initialisation.
- On the other hand, D executes two random generations (T_{rand}) and one PUF operation (T_{puf}).

Hence, the total computational cost incurred per node enrolment amounts to $2T_{\text{puf}}+2T_{\text{hash}}+4T_{\text{rand}}$. As shown in Table III, the proposed scheme achieves an enrolment cost of 0.18 ms, incurring a marginal overhead of 0.03 ms over DAuth [8] (0.15 ms). This cost is directly attributable to the additional hash operation for pseudonym initialisation — a security feature absent from DAuth. The scheme remains competitive with Zhou et al. [15] (0.13 ms), while additionally providing PUF-based secret binding and pseudonym initialisation at registration.

TABLE III COMPUTATIONAL COST COMPARISON: ENROLMENT PHASE.

Scheme	Operations	Cost (ms)
LAAKA [14]	$3T_{\mathsf{hash}}$	0.11
Zhou et al. [15]	$T_{\mathsf{puf}} + 2T_{\mathsf{hash}} + 3T_{\mathsf{rand}}$	0.13
DAuth [8]	$2T_{\mathsf{puf}} + 1T_{\mathsf{hash}} + 4T_{\mathsf{rand}}$	0.15
Proposed	$2T_{\mathsf{puf}} + 2T_{\mathsf{hash}} + 4T_{\mathsf{rand}}$	**0.18**

Benchmark (avg. 100 iterations): T_{rand}=0.0018 ms, T_{puf}=0.0522 ms, T_{hash}=0.0353 ms, T_{aes}=0.0867 ms.

Due to the execution of the authentication and key exchange phase, analysis of its computational efficiency is essential. In the proposed scheme, D and AS (Figures 2 and 3) both execute one PUF operation (T_{puf}).

- D additionally performs six hash operations (T_{hash}): pseudonym computation, unique secret hash, and XOR mask in the Authentication Phase, followed by mask recovery, session key derivation, and pseudonym rotation in the Key Exchange Phase.
- AS performs five hash operations (T_{hash}), along with one AES encryption (T_{aes}) for the gateway token and one random generation (T_{rand}) for m_{new}.
- GW runs one AES decryption (T_{aes}) to recover the session key and updated pseudonym from the forwarded token.

This leads to the total computational cost of the authentication and key exchange phase amounting to $2T_{\mathsf{puf}} + 11T_{\mathsf{hash}} + 2T_{\mathsf{aes}} + 1T_{\mathsf{rand}}$. In Table IV, the proposed scheme achieves a computational cost of 0.67 ms. It incurs a modest overhead of 0.11 ms over DAuth [8] (0.56 ms), directly attributable to three additional hash operations for pseudonym computation, dual-state lookup, and pseudonym rotation — the computational price of anonymity and desynchronisation resilience. The proposed scheme similarly incurs a modest overhead over LAAKA [14] (0.56 ms) and Zhou et al. [15] (0.53 ms), which neither employ PUF-based authentication nor a decoupled distributed authentication server.

TABLE IV COMPUTATIONAL COST COMPARISON: AUTHENTICATION & KEY EXCHANGE PHASE.

Scheme	Operations	Cost (ms)
LAAKA [14]	$16T_{\mathsf{hash}}$	0.56
Zhou et al. [15]	$15T_{\mathsf{hash}}$	0.53
DAuth [8]	$2T_{\mathsf{puf}} + 8T_{\mathsf{hash}} + 2T_{\mathsf{aes}} + 1T_{\mathsf{rand}}$	0.56
Proposed	$2T_{\mathsf{puf}} + 11T_{\mathsf{hash}} + 2T_{\mathsf{aes}} + T_{\mathsf{rand}}$	**0.67**

Benchmark (avg. 100 iterations): T_{rand}=0.0018 ms, T_{puf}=0.0522 ms, T_{hash}=0.0353 ms, T_{aes}=0.0867 ms.

2) Communication Cost: Communication cost is a core feature for analysing the performance of an authentication scheme, based on the number of message exchanges and their sizes (in bits). By assuming that communications are executed with uniform bit lengths, the communication cost of both enrolment and the authentication and key exchange phase is evaluated and compared with other state-of-the-art schemes.

It is assumed that identities and timestamps are 32 bits, along with nonces, hashes, challenges, responses, and ciphertexts as 256 bits, ensuring a fair comparison.

During the node enrolment phase, D enrols itself with AS, executing three message exchanges. The first message communicates the unique identity of D, i.e., ID_D (32 bits), to AS over the secure channel. In the second message, AS transmits a challenge c_D (256 bits) and a session-based random m_D (256 bits) to D. The third communication takes place from D to AS in the form of a unique secret y_D (256 bits), a PUF response R_D (256 bits), and a reverse challenge $c_{AS\text{-}D}$ (256 bits). Thus, the total communication cost per enrolment is 1312 bits as presented in Table V. It outperforms LAAKA [14] (2592 bits) and Zhou et al. [15] (1344 bits), and matches DAuth [8] (1312 bits) exactly, demonstrating that pseudonym initialisation adds no communication overhead over the base scheme. Unlike LAAKA and Zhou, the proposed scheme binds device enrolment to a PUF-derived secret and initialises the pseudonym at registration, adding security guarantees without increasing communication cost.

TABLE V COMMUNICATION COST COMPARISON: ENROLMENT PHASE.

Scheme	# Messages	Cost (bits)
LAAKA [14]	3	2592
Zhou et al. [15]	5	1344
DAuth [8]	3	1312
Proposed	3	**1312**

IDs/timestamps = 32 b; hashes/nonces/challenges/ciphertexts = 256 b.

TABLE VI COMMUNICATION COST COMPARISON: AUTH. & KEY EXCHANGE PHASE.

Scheme	# Messages	Cost (bits)
LAAKA [14]	3	2400
Zhou et al. [15]	4	2464
DAuth [8]	3	928
Proposed	3	**1376**

The authentication and key exchange phase is executed using three message communications. The first message takes place from D to AS, where D communicates its current pseudonym PID (256 bits), a masked authentication tag $Y^H_{AS\text{-}D}$ (256 bits), and a timestamp ts_1 (32 bits). In the second message, AS transmits the masked session seed m_H (256 bits) and a timestamp ts_2 (32 bits) to D. The third communication takes place from AS to GW, consisting of the updated pseudonym PID_{new} (256 bits), the identity ID_{AS} (32 bits), and the encrypted gateway token $\mathsf{auth\_token}_D$ (256 bits). Thus, the total communication cost becomes 1376 bits, as presented in Table VI. DAuth [8] requires 928 bits, with the 448-bit increase in the proposed scheme directly reflecting the widening of the device identifier from a 32-bit ID_D to a 256-bit pseudonym PID, the direct cost of adding device anonymity to all open-channel communications.

B. Simulation-Based Performance Analysis

Simulation-based performance analysis is important as it evaluates the proposed authentication scheme by considering the impact of message communication on performance metrics, such as energy and CPU time, in a multihop network.

A comprehensive simulation-based performance analysis is performed, assessing the performance efficiency of the proposed scheme against DAuth [8], LAAKA [14], and Zhou et al. [15]. The simulation is executed in the COOJA simulator using Cooja mote inside the Contiki-NG environment, where the CoAP protocol is utilized for the communication. A 100-node network is randomly deployed as the simulation scenario. The simulation is repeated ten times with random seeds. The propagation model in the simulation is UDGM (Distance Loss). In this network, 80 nodes are deployed as CoAP servers. One of these 80 nodes is considered to be the gateway or the root, depending on its role, which varies across the authentication schemes implemented. Out of the remaining 79 CoAP servers, a subset operates as authenticators. The other 20 nodes of the network are considered to be newly joined devices, functioning as CoAP clients. This consideration is conducted to introduce simultaneous authentication requests on the authenticator without congesting the network. Three complementary studies are conducted. The first is an overall per-device comparison with two authentication servers active. The second is an authentication server pool-size study that varies the number of active authentication servers from 2 to 10. The third is a network scalability study that grows the total node count across $N \in \{30, 50, 80, 100, 120\}$. All results are averaged over ten independent random seeds. Table VII elaborates the common simulation parameters.

TABLE VII SIMULATION PARAMETERS.

Parameter	Value
Operating System	Contiki-NG
Mote Type	Cooja Mote
Network Size	100 Nodes
Propagation Model	UDGM (Distance Loss)
Application Layer	CoAP
Network Layer Protocol	RPL Lite
MAC Layer Protocol	CSMA
Simulation Time	75 Seconds
No. of Simulation	10 times

1) Performance Analysis for Overall Per-Device Cost: Initially, the authentication schemes are evaluated by considering the overhead of one complete authentication cycle (enrolment, authentication, and key exchange) per device. For 20 newly joined nodes in a 100-node network with one gateway/root and two authentication servers in use, the proposed scheme demonstrated a consistent advantage over LAAKA [14] and Zhou et al. [15] by achieving a reduction of 42.8 % and 32.4 % with respect to per-device energy consumption (Figure 5a), respectively. Similarly, as shown in Figure 5(b), a reduction of 42.8 % and 32.4 % is demonstrated with respect to per-device CPU time over LAAKA and Zhou et al., respectively. The dominant savings arise in the authentication and key exchange phase. Due to the incorporation of fewer message exchanges and computationally lightweight hash-XOR operations, the proposed scheme has minimised radio activity and computational load compared to the message-intensive and cryptographically heavier LAAKA and Zhou et al. schemes. For Zhou et al., the reported costs comprise the enrolment and authentication phases only, as that scheme folds session-key agreement into its authentication exchange instead of executing a separate key-exchange round, whereas the remaining schemes include all three phases.

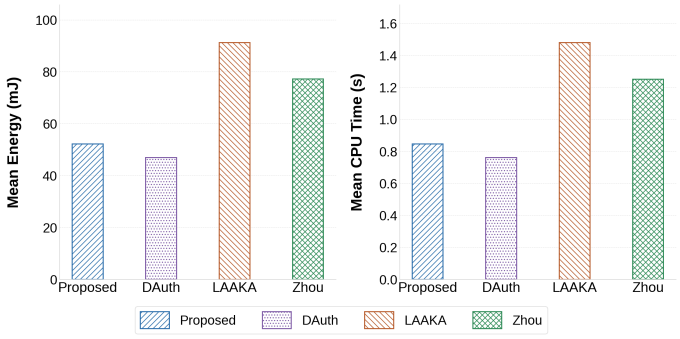
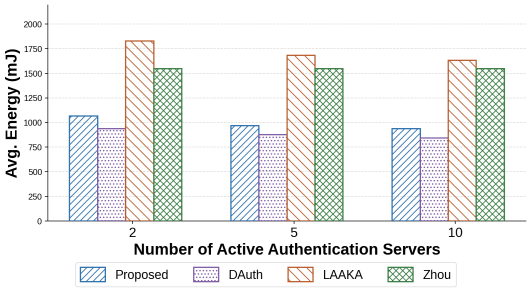


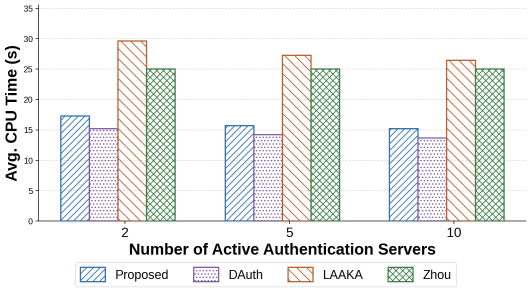
Fig. 5. Per-device Average cost, all phases : (a) Energy and (b) CPU time.

2) Performance Analysis for Authentication Server Pool Size: In the decoupled design, the gateway/root can assign any capable node as the authentication server, so the size of the authentication server pool directly governs how the authentication load is distributed. To isolate the benefit of decoupling rather than the raw per-device cost, each scheme is evaluated as the authentication server pool grows from 2 to 10 servers. Considering the total energy consumption (Figure 6a), the proposed scheme demonstrated a consistent advantage over LAAKA, achieving a reduction of approximately 42 % at every pool size, and the same advantage is demonstrated for total CPU time in Figure 6(b). Both distributed schemes also scale favourably with the pool, their total cost falling by roughly 11–12 % as the pool grows from 2 to 10 servers, since additional servers absorb the join load of the newly joined devices. Zhou et al., included as a gateway-coupled baseline, has no authentication server pool and its cost remains essentially unchanged across pool sizes, confirming that a centralised design cannot convert authentication server diversity into load relief. These consistent reductions demonstrate the energy and computational efficiency of the decoupled authentication server design in the proposed scheme.

3) Performance Analysis for Network Scalability: The performance of the authentication schemes is also analysed by considering their scalability with respect to total network size N, varied from 30 to 120 nodes, where 20 % of the nodes in each network are newly joined devices. Considering the total energy consumption (Figure 7a), at the maximum tested network size $N = 120$, the proposed scheme demonstrates a reduction of 42.7 % over LAAKA and 37.6 % over Zhou et al. Similarly, as shown in Figure 7(b), a reduction of 42.7 % and 37.6 % is demonstrated with respect to total CPU time over LAAKA and Zhou et al., respectively, at $N = 120$. LAAKA and Zhou et al. exhibit steeper energy and CPU growth with increasing N because their heavier per-device cryptographic load — AES-intensive authentication in LAAKA and a combined heavy authentication and key exchange in Zhou et al. — accumulates faster as the device count grows. Due to

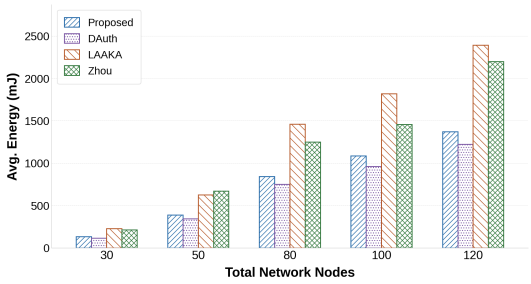


(a)

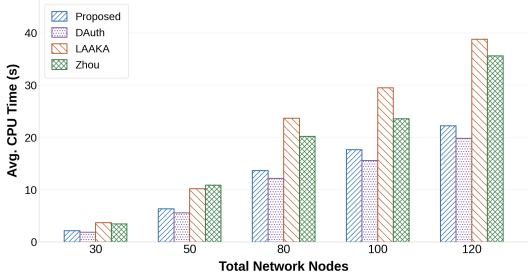


(b)

Fig. 6. Average authentication overhead vs. number of active authentication servers (20 devices): (a) energy and (b) CPU time.



(a)



(b)

Fig. 7. Network scalability: (a) Avg. energy and (b) Avg. CPU time vs. network size N.

the incorporation of lightweight XOR-hash operations and a decoupled authentication server design, the proposed scheme keeps per-device cost low and achieves the lowest total cost among all anonymity-preserving schemes (LAAKA and Zhou et al.) across all measured network sizes, while incurring only a modest overhead above DAuth attributable to the additional pseudonym-rotation mechanism. These consistent reductions across $N = 30$ to 120 demonstrate the scalability and computational efficiency of the proposed scheme in comparison to the benchmark schemes.

4) Performance Analysis for Desynchronisation Recovery: During the Key Exchange Phase, the authentication server simultaneously dispatches two packets: one to the gateway/root carrying $(PID_{\text{new}}, ID_{AS}, \texttt{auth\_token}_D)$, and one to the device carrying (m_H, ts_2). Desynchronisation arises when the packet destined for the device is lost in transit, for any reason. After such a loss, the device retains its old state m_{curr} while the authentication server has already rotated to a new state, causing subsequent authentication attempts to fail.

DAuth [8] stores only a single nonce state at the authentication server. When the device re-authenticates using the old m_{curr}, the authentication server has no means to recognise it and rejects the request outright. The device must therefore undergo a full re-enrolment (Phase 1: PUF challenge-response and accumulator setup) before a new authentication and key exchange can succeed. The proposed scheme eliminates this penalty through its dual-state mechanism: the authentication server retains both $(m_{\text{curr}}, PID_{\text{curr}})$ and $(m_{\text{old}}, PID_{\text{old}})$. When the device arrives with the old pseudonym, the authentication server recognises it via the PID_{old} lookup and runs the full authentication protocol: recovering the PUF response R_D, deriving $Y_{AS\text{-}D}^H$ from the device's unique secret y_D, and verifying the accumulator using m_{old}, then completing a fresh key exchange, all without re-enrolment.

Since the dual-state desynchronisation recovery mechanism is a direct extension of DAuth's single-state protocol, only DAuth serves as the meaningful baseline for this comparison. LAAKA and Zhou et al. employ fundamentally different authentication architectures with no equivalent session-state nonce rotation, making a like-for-like desynchronisation comparison inapplicable.

Figure 8 presents a per-device energy and CPU time comparison of a single authentication round under two conditions: normal operation (before any packet loss) and recovery (the round immediately following a desynchronisation event), evaluated over a 100-node multi-hop RPL network with 20 devices and 5 independent seeds. As shown in Figure 8(a), DAuth [8] incurs a 118.3 % increase in per-device energy on a recovery round over its normal round, reflecting the overhead of a failed authentication attempt followed by full re-enrolment. In contrast, the proposed scheme's recovery round is statistically indistinguishable from its normal round, confirming that the dual-state mechanism incurs negligible additional overhead during recovery. Compared directly, the proposed scheme reduces per-device recovery energy by 35.3 % over DAuth [8]. The CPU time results in Figure 8(b) exhibit the same pattern, yielding a 35.3 % reduction in execution time over DAuth [8].

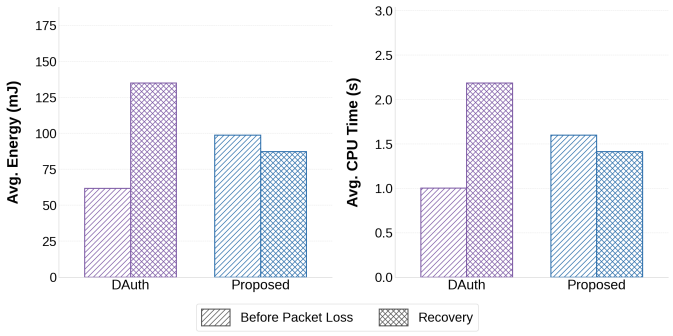


Fig. 8. Per-device desynchronisation recovery cost: (a) energy and (b) CPU time for a normal round vs. a recovery round, averaged over 20 devices in a 100-node network.

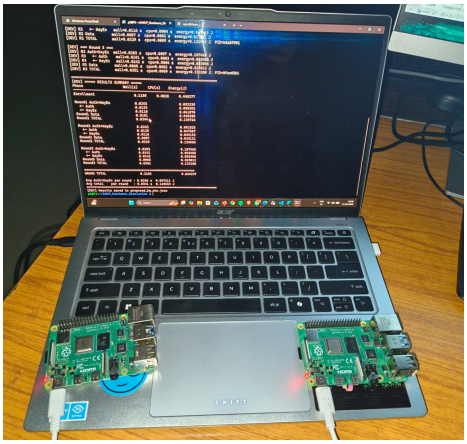


Fig. 9. Hardware testbed: two Raspberry Pi 4B boards (left: authentication server; right: device) connected to a Windows laptop (gateway).

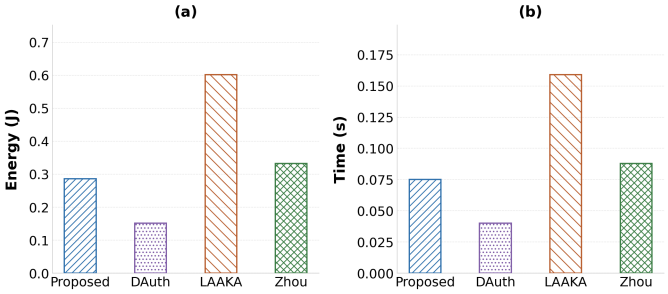


Fig. 10. Hardware-based performance:Avg. Energy (a) and CPU time (b) on Raspberry Pi 4B.

C. Hardware-Based Performance Analysis

Hardware-based performance analysis plays a crucial role as it validates the feasibility of the proposed authentication scheme by implementing it on physical IoT devices, incorporating real hardware environment conditions. The proposed scheme and the three comparison schemes—DAuth [8], LAAKA [14], and Zhou et al. [15]—are implemented on a Raspberry Pi 4B platform, equipped with a quad-core 64-bit ARM Cortex-A72 processor operating at 1.5 GHz. A uniform hardware setup is maintained across all compared authentication schemes, ensuring a fair and consistent comparison of energy consumption and CPU time. The implementation considers two Raspberry Pi 4B platforms as IoT nodes. One performs the role of the newly joined device, initiating the authentication process. The other functions as the authentication server, performing the necessary verification for authentication and session key exchange. A Windows laptop is leveraged as the gateway, handling authentication verification and session key delivery. Each of the three entities maintains direct TCP communication with the others. Energy consumption is estimated using the active power profile of the Raspberry Pi 4B (3.8 W) multiplied by the measured wall-clock time per authentication round, capturing both cryptographic computation and network round-trip latency. The physical testbed is shown in Figure 9.

As illustrated in Figure 10(a), DAuth achieves the lowest per-round energy of 0.152 J, as it performs no pseudonym operations. The proposed scheme consumes 0.286 J per round—an overhead of approximately 88 % over DAuth attributable to the additional SHA-256 evaluations for PID rotation and dual-state verification. Despite this overhead, the proposed scheme remains 14.2 % and 52.5 % more energy-efficient than Zhou et al. (0.333 J) and LAAKA (0.602 J), respectively, confirming that the anonymity guarantee is achieved at a cost well within the range of existing non-anonymous schemes.

The CPU time results in Figure 10(b) follow the same ordering: DAuth (0.040 s), Proposed (0.075 s), Zhou et al. (0.088 s), and LAAKA (0.159 s). The proposed scheme reduces CPU time by 14.8 % and 52.8 % relative to Zhou et al. and LAAKA, respectively, while incurring an 87.5 % overhead compared to DAuth—consistent with the energy results. These reductions are consistent with the lightweight XOR-hash operations and decoupled authentication server design of the proposed scheme, which translate directly to measurable gains in physical energy and execution time on real hardware. The inconsistency between hardware and simulation-based results arises due to the disparity in scale, where a 100-node network is assumed in simulation whereas the hardware setup involves only two Raspberry Pi platforms.

VI. Conclusion

This paper presented a lightweight, anonymous, and decoupled distributed authentication scheme for multihop IoT networks. The assigned authentication server, which may or may not be the parent of the newly joined device, performs authentication (Authentication Phase) and session key establishment (Key Exchange Phase) in sequential phases. Device anonymity is preserved through per-session pseudonym rotation, preventing adversarial tracking across sessions. A dual-state mechanism at both the authentication server and the gateway provides transparent desynchronisation recovery without re-enrolment. PUF-based authentication eliminates long-term secret storage, and hash-based accumulator operations with XOR masking keep the computational overhead low.

Formal verification in ProVerif confirms authentication correspondence, session key secrecy, resistance to offline guessing attacks, and observational-equivalence-based anonymity. COOJA/Contiki-NG simulations confirm practical suitability. The proposed scheme achieves a per-device three-phase energy of 52.19 mJ, reducing total energy by 42.8 % over LAAKA [14] and 32.4 % over Zhou et al. [15], with consistent gains across varying network sizes and authentication server pool sizes. Hardware evaluation on an RPi 4B testbed corroborates these findings: the proposed scheme consumes 0.286 J per authentication round, achieving 14.2 % and 52.5 % energy savings over Zhou et al. and LAAKA, respectively.

REFERENCES

[1] I. Zhou, I. Makhdoom, N. Shariati, M. A. Raza, R. Keshavarz, J. Lipman, M. Abolhasan, and A. Jamalipour, "Internet of things 2.0: Concepts, applications, and future directions," *IEEE Access*, vol. 9, pp. 70 961–71 012, 2021.

[2] N. Lajnef, A. Mami, and F. Derbel, "Applications of wireless sensor networks and IoT frameworks in industry 4.0: A systematic literature review," *Sensors*, vol. 22, no. 6, 2022.

[3] M. Khatua, S. Das, and P. M. Rana, "PUF-based lightweight mutual authentication scheme for IoT-based smart grids," in *Proc. IEEE ANTS*. IEEE, 2024, pp. 1–6.

[4] S. A. Sheik and A. P. Muniyandi, "Secure authentication schemes in cloud computing with glimpse of artificial neural networks: A review," *Cyber Security and Applications*, vol. 1, p. 100002, 2023.

[5] D. He, Y. Cai, S. Zhu, Z. Zhao, S. Chan, and M. Guizani, "A lightweight authentication and key exchange protocol with anonymity for IoT," *IEEE Trans. Wireless Communications*, vol. 22, no. 11, pp. 7862–7872, 2023.

[6] S. Roy, M. H. Mahalat, and B. Sen, "Design and implementation of cost-effective end-to-end authentication protocol for PUF-enabled IoT devices," *IEEE Trans. Emerging Topics in Computing*, 2025.

[7] P. Rosa, A. Souto, and J. Cecílio, "Light-SAE: A lightweight authentication protocol for large-scale IoT environments made with constrained devices," *IEEE Trans. Network and Service Management*, vol. 20, no. 3, pp. 2428–2441, 2023.

[8] S. Das and M. Khatua, "Lightweight and decoupled distributed authentication for multihop IoT networks," in *Proc. IEEE COMSNETS*, 2026.

[9] O. Alruwaili, M. Tanveer, S. A. Aldossari, S. Alanazi, and A. Armghan, "RAAF-MEC: Reliable and anonymous authentication framework for IoT-enabled mobile edge computing environment," *Internet of Things*, vol. 29, p. 101459, 2025.

[10] Y. Zheng, W. Liu, C. Gu, and C.-H. Chang, "PUF-based mutual authentication and key exchange protocol for peer-to-peer IoT applications," *IEEE Trans. Dependable and Secure Computing*, vol. 20, no. 4, pp. 3299–3316, 2022.

[11] S. Li, T. Zhang, B. Yu, and K. He, "A provably secure and practical PUF-based end-to-end mutual authentication and key exchange protocol for IoT," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 5487–5501, 2020.

[12] W. Yang, C. Hou, Y. Wang, Z. Zhang, X. Wang, and Y. Cao, "SAKMS: A secure authentication and key management scheme for IETF 6TiSCH industrial wireless networks based on improved elliptic-curve cryptography," *IEEE Trans. Network Science and Engineering*, vol. 11, no. 3, pp. 3174–3188, 2024.

[13] J. Zhong, S. He, Z. Liu, and L. Xiong, "Lightweight anonymous authentication for IoT: A taxonomy and survey of security frameworks," *Sensors*, vol. 25, no. 17, p. 5594, 2025.

[14] H. Ali and I. Ahmed, "LAAKA: A lightweight anonymous authentication and key agreement scheme for IoT-fog environment," *Computers & Security*, vol. 140, p. 103770, 2024.

[15] X. Zhou, S. Wang, K. Wen, B. Hu, X. Tan, and Q. Xie, "Security-enhanced lightweight and anonymity-preserving user authentication scheme for IoT-based healthcare," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 9718–9731, 2024.

[16] B. Blanchet, B. Smyth, V. Cheval, and M. Sylvestre, "ProVerif 2.00: Automatic cryptographic protocol verifier, user manual and tutorial," INRIA, Tech. Rep., 2018.

[17] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.